

Clean Comments

www.cs.uoi.gr/~zarras/soft-devII.htm

from Clean Code by R. C. Martin, a.k.a "Uncle Bob"

1

Comments - A necessary evil !!

The proper use of comments is to **compensate** for our **failure** to **express** ourselves in code !!!

We **must have them** because we cannot always figure out how to express ourselves without them, but their use is **not** a cause for **celebration**....

2

Comments don't make up for bad code !!

One of the more common **motivations** for writing **comments** is **bad code**.

We write a module and we know it is confusing and disorganized. We know it's a mess. So we say to ourselves, "Ooh, I'd better comment that!"

Rather than spend your time writing the comments that explain the mess you've made, spend it **cleaning that mess**.

3

Explain yourself in code !!

```
// Check to see if the employee is eligible for full benefits  
if ((employee.flags & HOURLY_FLAG) && (employee.age > 65))  
.....
```

A comment over a complex expression

Can we do better than this ??

4

Explain yourself in code !!

```
// Check to see if the employee is eligible for full benefits
if ((employee.flags & HOURLY_FLAG) && (employee.age > 65))
.....
```

A comment over a complex expression

Can we do better than this ??

It takes only a few seconds of thought to explain most of our intent in code.

In many cases it's simply a matter of **creating a function** that **says** the same thing as the **comment** you want to write.

```
if (employee.isEligibleForFullBenefits())
```

5

Comments lie !!

```
MockRequest request;
private final String HTTP_DATE_REGEX =
    "[SMTWF][a-z]{2}\\s[0-9]{2}\\s[JFMASOND][a-z]{2}\\s"+
    "[0-9]{4}\\s[0-9]{2}\\s:[0-9]{2}\\s:[0-9]{2}\\sGMT";
private Response response;
private FitNesseContext context;
private FileResponder responder;
private Locale saveLocale;
// Example: "Tue, 02 Apr 2003 22:18:49 GMT"
```

Do you see anything strange ??

6

Comments lie !!

```
MockRequest request;
private final String HTTP_DATE_REGEX =
    "[SMTWF][a-z]{2}\\,\\s[0-9]{2}\\s[JFMASOND][a-z]{2}\\s"+
    "[0-9]{4}\\s[0-9]{2}\\:[0-9]{2}\\:[0-9]{2}\\sGMT";
private Response response;
private FitNesseContext context;
private FileResponder responder;
private Locale saveLocale;
// Example: "Tue, 02 Apr 2003 22:18:49 GMT"
```

Comments lie !! Not always, and not intentionally, but too **often**.

The **older** a comment is, and the farther away it is from the code it describes, the more likely it is to be just plain **wrong**.

The reason is simple. Programmers **can't** realistically **maintain them**. Code changes and evolves. Chunks of it move from here to there. Those chunks bifurcate and reproduce and come together again to form chimeras. Unfortunately the comments don't always follow them—*can't always follow them*.

7



Good Comments !!

8

Legal Comments

```
// Copyright (C) 2003,2004,2005 by Object Mentor, Inc.
// All rights reserved.
// Released under the terms of the GNU
// General Public License version.
```

Copyright and **authorship** statements are necessary and reasonable things to put into a comment at the start of each source file.

BUT, comments like this should not be contracts or legal tomes.

For example, where possible, refer to a standard license or other external document rather than putting all the terms and conditions into the comment.

9

Informative Comments

It is sometimes useful to provide basic information **that is not present in the implementation** with a comment.

For example, this comment that explains the return value of an abstract method; the method **does not have an implementation** so we cannot **figure out** the intent of the **returned value**:

```
// Returns an instance of the Responder being tested.
protected abstract Responder responderInstance();
```

Could we avoid it ??

10

Informative Comments

It is sometimes useful to provide basic information that is not present in the implementation with a comment.

For example, this comment that explains the return value of an abstract method; the method does not have an implementation so there is no way to figure out the intent of the returned value:

```
// Returns an instance of the Responder being tested.
protected abstract Responder responderInstance();
```

Could we avoid it ??

The method is not implemented but a better name would do

```
protected abstract Responder getResponderBeingTested();
```

11

Explain design decisions

Sometimes a comment goes beyond just useful information about the implementation and **provides the intent behind a decision**.

In the following case, we see an interesting decision documented by a comment. When comparing any object that does not belong to the hosting class (`WikiPagePath`), with a `WikiPagePath` object, the latter is considered greater.

```
public int compareTo(Object o)
{
    if(o instanceof WikiPagePath)
    {
        WikiPagePath p = (WikiPagePath) o;
        String compressedName = StringUtil.join(names, "");
        String compressedArgumentName = StringUtil.join(p.names, "");
        return compressedName.compareTo(compressedArgumentName);
    }
    return 1; // we are greater because we are the right type.
```

without the comment we could wonder why, with the comment its clearer that this was **just a decision** taken by the **author**

12

Clarification of arguments, ret values

Sometimes it is just helpful to **clarify** the meaning of some obscure **argument** or **return value** into something that's readable.

```
public void testCompareTo() throws Exception
{
    WikiPagePath a = PathParser.parse("PageA");
    WikiPagePath ab = PathParser.parse("PageA.PageB");
    WikiPagePath b = PathParser.parse("PageB");
    WikiPagePath aa = PathParser.parse("PageA.PageA");
    WikiPagePath bb = PathParser.parse("PageB.PageB");
    WikiPagePath ba = PathParser.parse("PageB.PageA");

    assertTrue(a.compareTo(a) == 0);    // a == a
    assertTrue(a.compareTo(b) != 0);    // a != b
    assertTrue(ab.compareTo(ab) == 0);  // ab == ab
    assertTrue(a.compareTo(b) == -1);   // a < b
    assertTrue(aa.compareTo(ab) == -1); // aa < ab
    assertTrue(ba.compareTo(bb) == -1); // ba < bb
    assertTrue(b.compareTo(a) == 1);    // b > a
    assertTrue(ab.compareTo(aa) == 1);  // ab > aa
    assertTrue(bb.compareTo(ba) == 1);  // bb > ba
}
```

Useful but hard to verify that comments are correct.

Are they ??

13

Warnings

Sometimes it is useful to **warn** other programmers about certain **consequences**.

```
// Don't run unless you
// have some time to kill.
public void _testWithReallyBigFile(){
    writeLinesToFile(10000000);
    response.setBody(testFile);
    response.readyToSend(this);
    String responseString = output.toString();
    assertSubString("Content-Length: 1000000000", responseString);
    assertTrue(bytesSent > 1000000000);
}
```

14

TODO

It is sometimes reasonable to leave “To do” notes in the form of //TODO comments.

In the following case, the TODO comment explains why the function has a degenerate implementation and what that function’s future should be.

```
//TODO-MdM these are not needed
// We expect this to go away when we do the checkout model
protected VersionInfo makeVersion() throws Exception {
    return null;
}
```

TODOs are **jobs** that the programmer thinks **should be done**, but for some reason can’t do at the moment.

It might be a **reminder** to delete a **deprecated** feature or a plea for someone else to look at a problem.

It might be a **request** for someone else to think of a better name or a reminder to make a change.

15

Amplification

A comment may be used to amplify the importance of something that may otherwise seem inconsequential.

```
String listItemContent = match.group(3).trim();
// the trim is real important. It removes the starting
// spaces that could cause the item to be recognized
// as another list.
new ListItemWidget(this, listItemContent, this.level + 1);
return buildList(text.substring(match.end()));
```

16

Javadocs

There is nothing quite so helpful and satisfying as a **well-described public API**. The **javadocs** for the standard Java library are a case in point.

It would be difficult, at best, to write Java programs without them.

If we are writing a public API, then we should certainly write good javadocs for it.

But keep that Javadocs can be just as misleading, nonlocal, and dishonest as any other kind of comment.

```
/**
 * This class demonstrates documentation comments.
 * @author Ayan Ahmed
 * @version 1.2
 */
public class SquareNum {
    /**
     * This method returns the square of num.
     * This is a multiline description. You can use
     * as many lines as you like.
     * @param num The value to be squared.
     * @return num squared.
     */
    public double square(double num) {
        return num * num;
    }
    /**
     * This method inputs a number from the user.
     * @return The value input as a double.
     * @exception IOException On input error.
     * @see IOException
     */
    public double getNumber() throws IOException {
        InputStreamReader isr = new InputStreamReader(System.in);
        BufferedReader inData = new BufferedReader(isr);
        String str;
        str = inData.readLine();
        return (new Double(str)).doubleValue();
    }
    /**
     * This method demonstrates square().
     * @param args Unused.
     * @return Nothing.
     * @exception IOException On input error.
     * @see IOException
     */
    public static void main(String args[]) throws IOException
    {
        SquareNum ob = new SquareNum();
        double val;
        System.out.println("Enter value to be squared: ");
        val = ob.getNumber();
        val = ob.square(val);
        System.out.println("Squared value is " + val);
    }
}
```

17

Bad Comments !!

18

Mumbling

```
public void loadProperties()
{
    try
    {
        String propertiesPath = propertiesLocation + "/" + PROPERTIES_FILE;
        FileInputStream propertiesStream = new FileInputStream(propertiesPath);
        loadedProperties.load(propertiesStream);
    }
    catch(IOException e)
    {
        // No properties files means all defaults are loaded
    }
}
```

What does that comment in the catch block mean?

Clearly it meant something to the author, but the meaning does not come through all that well.

19

Mumbling

```
public void loadProperties()
{
    try
    {
        String propertiesPath = propertiesLocation + "/" + PROPERTIES_FILE;
        FileInputStream propertiesStream = new FileInputStream(propertiesPath);
        loadedProperties.load(propertiesStream);
    }
    catch(IOException e)
    {
        // No properties files means all defaults are loaded
    }
}
```

MAYBE, if we get an IOException, it means that there was no properties file; and in that case all the defaults are loaded.

But are they loaded in a different place? **Where**? **Who** is responsible for that?

20

Mumbling

```
public void loadProperties()
{
    try
    {
        String propertiesPath = propertiesLocation + "/" + PROPERTIES_FILE;
        FileInputStream propertiesStream = new FileInputStream(propertiesPath);
        loadedProperties.load(propertiesStream);
    }
    catch(IOException e)
    {
        // No properties files means all defaults are loaded
    }
}
```

Or — and this is the **scary possibility** —the author trying to tell himself to come back here later and **write the code** that would load the defaults?

21

Mumbling

```
public void loadProperties()
{
    try
    {
        String propertiesPath = propertiesLocation + "/" + PROPERTIES_FILE;
        FileInputStream propertiesStream = new FileInputStream(propertiesPath);
        loadedProperties.load(propertiesStream);
    }
    catch(IOException e)
    {
        // No properties files means all defaults are loaded
    }
}
```

Our only recourse is to examine the code in other parts of the system to **find out what's going on**.

Any comment that forces you to look in another module for the meaning of that comment has failed to communicate to you and is not worth the bits it consumes !!

22

Redundant Comments

Listing 4-1

waitForClose

```
// Utility method that returns when this.closed is true. Throws an exception
// if the timeout is reached.
public synchronized void waitForClose(final long timeoutMillis)
throws Exception
{
    if(!closed)
    {
        wait(timeoutMillis);
        if(!closed)
            throw new Exception("MockResponseSender could not be closed");
    }
}
```

What purpose does this comment serve?

23

Redundant Comments

Listing 4-1

waitForClose

```
// Utility method that returns when this.closed is true. Throws an exception
// if the timeout is reached.
public synchronized void waitForClose(final long timeoutMillis)
throws Exception
{
    if(!closed)
    {
        wait(timeoutMillis);
        if(!closed)
            throw new Exception("MockResponseSender could not be closed");
    }
}
```

It's certainly **not more informative** than the **code**.

It is **not easier** to read than the code.

Indeed, it is **less precise** than the code and entices the reader to accept that lack of precision in lieu of true understanding.

Probably it **takes longer** to read than the code itself !!!

24

Redundant Comments

Listing 4-2

ContainerBase.java (Tomcat)

```
public abstract class ContainerBase
    implements Container, Lifecycle, Pipeline,
    MBeanRegistration, Serializable {

    /**
     * The processor delay for this component.
     */
    protected int backgroundProcessorDelay = -1;

    /**
     * The lifecycle event support for this component.
     */
    protected LifecycleSupport lifecycle =
        new LifecycleSupport(this);

    /**
     * The container event listeners for this Container.
     */
    protected ArrayList listeners = new ArrayList();

    /**
     * The Loader implementation with which this Container is
     * associated.
     */
    protected Loader loader = null;

    /**
     * The Logger implementation with which this Container is
     * associated.
     */
    protected Log logger = null;

    /**
     * Associated logger name.
     */
    protected String logName = null;
```

More **useless** and **redundant** javadoc comments in a class taken from Tomcat.

These comments serve only to clutter and obscure the code.

Remember → vertical density !!

25

Redundant Comments

```
/** The name. */
private String name;

/** The version. */
private String version;

/** The licenceName. */
private String licenceName;

/** The version. */
private String info;
```

more **redundant/noise** comments !!

But there is something even more **scary** here !!

can you spot it ??

26

Redundant Comments

```
/** The name. */
private String name;

/** The version. */
private String version;

/** The licenceName. */
private String licenceName;

/** The version. */
private String info;
```

more **redundant/noise** comments !!

But there is something even more **scary** here !!

can you spot it ??

there is a copy paste error in the last comment !!

27

Misleading Comments

Listing 4-1

waitForClose

```
// Utility method that returns when this.closed is true. Throws an exception
// if the timeout is reached.
public synchronized void waitForClose(final long timeoutMillis)
throws Exception
{
    if(!closed)
    {
        wait(timeoutMillis);
        if(!closed)
            throw new Exception("MockResponseSender could not be closed");
    }
}
```

Sometimes, with all the best intentions, a programmer makes a statement in his comments that isn't precise enough to be accurate.

Do you see how the above comment is misleading?

28

Misleading Comments

Listing 4-1

waitForClose

```
// Utility method that returns when this.closed is true. Throws an exception
// if the timeout is reached.
public synchronized void waitForClose(final long timeoutMillis)
    throws Exception
{
    if(!closed)
    {
        wait(timeoutMillis);
        if(!closed)
            throw new Exception("MockResponseSender could not be closed");
    }
}
```

Sometimes, with all the best intentions, a programmer makes a statement in his comments that **isn't precise** enough to be accurate.

Do you see how the above comment is misleading?

The method does not return when closed is true ; it returns if closed is true, else it waits for a time out and if closed is still false it throws an exception.

29

Mandated Comments

Listing 4-3

```
/**
 *
 * @param title The title of the CD
 * @param author The author of the CD
 * @param tracks The number of tracks on the CD
 * @param durationInMinutes The duration of the CD in minutes
 */
public void addCD(String title, String author,
                  int tracks, int durationInMinutes) {
    CD cd = new CD();
    cd.title = title;
    cd.author = author;
    cd.tracks = tracks;
    cd.duration = duration;
    cdList.add(cd);
}
```

How about these comments ??

These comments are there because of a rule followed by the development team that requires **each function** to have a javadoc **comment**

30

Mandated Comments

Listing 4-3

```
/**
 *
 * @param title The title of the CD
 * @param author The author of the CD
 * @param tracks The number of tracks on the CD
 * @param durationInMinutes The duration of the CD in minutes
 */
public void addCD(String title, String author,
                  int tracks, int durationInMinutes) {
    CD cd = new CD();
    cd.title = title;
    cd.author = author;
    cd.tracks = tracks;
    cd.duration = duration;
    cdList.add(cd);
}
```

How about these comments ??

These comments are there because of a rule followed by the development team that requires **each function** to have a javadoc **comment**

It is just plain silly to have a rule that says that every function must have a javadoc, or every variable must have a comment !!

31

Log Comments

```
* Changes (from 11-Oct-2001)
* -----
* 11-Oct-2001 : Re-organised the class and moved it to new package
*               com.jrefinery.date (DG);
* 05-Nov-2001 : Added a getDescription() method, and eliminated NotableDate
*               class (DG);
* 12-Nov-2001 : IBD requires setDescription() method, now that NotableDate
*               class is gone (DG); Changed getPreviousDayOfWeek(),
*               getFollowingDayOfWeek() and getNearestDayOfWeek() to correct
*               bugs (DG);
* 05-Dec-2001 : Fixed bug in SpreadsheetDate class (DG);
* 29-May-2002 : Moved the month constants into a separate interface
*               (MonthConstants) (DG);
* 27-Aug-2002 : Fixed bug in addMonths() method, thanks to N???levka Petr (DG);
* 03-Oct-2002 : Fixed errors reported by Checkstyle (DG);
* 13-Mar-2003 : Implemented Serializable (DG);
* 29-May-2003 : Fixed bug in addMonths method (DG);
* 04-Sep-2003 : Implemented Comparable. Updated the isInRange javadocs (DG);
* 05-Jan-2005 : Fixed bug in addYears() method (1096282) (DG);
```

How about these comments ??

32

Log Comments

```
* Changes (from 11-Oct-2001)
*
* 11-Oct-2001 : Re-organised the class and moved it to new package
*               com.jrefinery.date (DG);
* 05-Nov-2001 : Added a getDescription() method, and eliminated NotableDate
*               class (DG);
* 12-Nov-2001 : IBD requires setDescription() method, now that NotableDate
*               class is gone (DG); Changed getPreviousDayOfWeek(),
*               getFollowingDayOfWeek() and getNearestDayOfWeek() to correct
*               bugs (DG);
* 05-Dec-2001 : Fixed bug in SpreadsheetDate class (DG);
* 29-May-2002 : Moved the month constants into a separate interface
*               (MonthConstants) (DG);
* 27-Aug-2002 : Fixed bug in addMonths() method, thanks to N???levka Petr (DG);
* 03-Oct-2002 : Fixed errors reported by Checkstyle (DG);
* 13-Mar-2003 : Implemented Serializable (DG);
* 29-May-2003 : Fixed bug in addMonths method (DG);
* 04-Sep-2003 : Implemented Comparable. Updated the isInRange javadocs (DG);
* 05-Jan-2005 : Fixed bug in addYears() method (1096282) (DG);
```

How about these comments ??

Long ago there was a good reason to create and maintain these log entries at the start of every module. We didn't have source code **control systems** that did it for us.

Nowadays, however, these long comments are just more clutter to obfuscate the module. They should be **completely removed**.

33

Position Markers

Sometimes programmers like to mark a particular position in a source file.

```
// Actions //////////////////////////////////////
```

There are **rare times** when it makes sense to gather certain functions together beneath a **banner** like this. But in general they are **clutter** that should be eliminated.

So use them very **sparingly**, and only when the benefit is significant.

34

Closing Brace Comments

Listing 4-6 (continued)

wc.java

```
while ((line = in.readLine()) != null) {
    lineCount++;
    charCount += line.length();
    String words[] = line.split("\\W");
    wordCount += words.length;
} //while
System.out.println("wordCount = " + wordCount);
System.out.println("lineCount = " + lineCount);
System.out.println("charCount = " + charCount);
} // try
catch (IOException e) {
    System.err.println("Error:" + e.getMessage());
} //catch
} //main
}
```

Closing brace comments make sense for **long functions** with deeply nested structures.

35

Closing Brace Comments

Listing 4-6 (continued)

wc.java

```
while ((line = in.readLine()) != null) {
    lineCount++;
    charCount += line.length();
    String words[] = line.split("\\W");
    wordCount += words.length;
} //while
System.out.println("wordCount = " + wordCount);
System.out.println("lineCount = " + lineCount);
System.out.println("charCount = " + charCount);
} // try
catch (IOException e) {
    System.err.println("Error:" + e.getMessage());
} //catch
} //main
}
```

Closing brace comments make sense for **long functions** with deeply nested structures.

BUT, we don't like long functions !!

So if you find yourself wanting to mark your closing braces, try to shorten your functions instead !!

36

Attributions

```
/* Added by Rick */
```

Source code control systems are very good at remembering who added what, when.

There is no need to pollute the code with little bylines.

You might think that such comments would be useful in order to help others know who to talk to about the code. But the reality is that they tend to stay around for years and years, getting **less and less accurate and relevant**.

37

Commented out code

```
InputStreamResponse response = new InputStreamResponse();
response.setBody(formatter.getResultStream(), formatter.getByteCount());
// InputStream resultsStream = formatter.getResultStream();
// StreamReader reader = new StreamReader(resultsStream);
// response.setContent(reader.read(formatter.getByteCount()));
```

**Few practices are as odious as commenting-out code.
Don't do this !!**

Others who see that commented-out code won't have the courage to delete it. They'll think it is there for a reason and is too **important to delete**.

38

What does this comment say ?

HTML in Comments

If comments are going to be extracted by some tool (like Javadoc) to appear in a Web page, then it should be the responsibility of that tool, and not the programmer, to adorn the comments with appropriate HTML.

20

Non local information

```
/**
 * Port on which fitnessse would run. Defaults to <b>8082</b>.
 *
 * @param fitnesssePort
 */
public void setFitnesssePort(int fitnesssePort)
{
    this.fitnesssePort = fitnesssePort;
}
```

How about this comment ??

41

Non local information

```
/**
 * Port on which fitnessse would run. Defaults to <b>8082</b>.
 *
 * @param fitnesssePort
 */
public void setFitnesssePort(int fitnesssePort)
{
    this.fitnesssePort = fitnesssePort;
}
```

Aside from the fact that it is horribly **redundant**, it also offers information about the **default port !!**

And yet the function has absolutely no control over what that default is. The comment is **not describing the function**, but some other, **far distant part of the system**.

Of course there is no guarantee that this comment will be changed when the code containing the default port is changed !!!

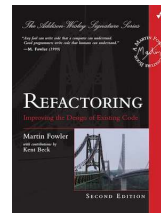
42

Javadocs in non public code

As useful as javadocs are for public APIs, they are anathema to code that is not intended for public consumption.

Generating javadoc pages for the classes and functions inside a system is not generally useful.

43



Smells and related refactorings

<https://refactoring.com/catalog/>

44

Comments

Symptoms

Scan the text for “//” or “/*” (comment markers).

Causes

Usually, for the best of reasons. The author realizes that **something isn't as clear** as it could be, and adds a comment.

Some comments are particularly **helpful**:

Legal comments, informative, Javadoc public API, TODO, amplification, warnings. Etc....

45

Comments

What to Do

When a comment explains a block of code, you can often use **Extract Method** to pull the block out into a separate method.

The comment will often suggest a **name** for the new method.

46

Extract Method

```
void printOwing() {
    printBanner();

    //print details
    System.out.println ("name:  " + _name);
    System.out.println ("amount " + getOutstanding());
}
```

If you have a **code fragment** that can be **grouped together**.

Turn the fragment into a method whose name explains the purpose of the method.



```
void printOwing() {
    printBanner();
    printDetails(getOutstanding());
}

void printDetails (double outstanding) {
    System.out.println ("name:  " + _name);
    System.out.println ("amount " + outstanding);
}
```

47

Extract Method

Extract Method is one of the most common refactorings

If you have a **code fragment** that can be **grouped together**.

Turn the fragment into a method whose name explains the purpose of the method.

Short, well-named methods increase the chances that other methods can **reuse** them.

It allows the higher-level methods to **read** more like a **story**.

Overriding also is easier when the methods are fine grained.

Small methods really work only when you have **good names**, so you need to pay attention to naming.

48

Comments

What to Do

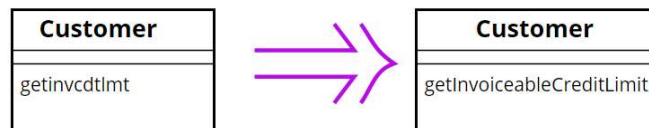
When a comment explains **what a method does** (better than the method's name!), use **Rename Method** using the comment as the basis of the new **name**.

49

Rename Method

The name of a method does not reveal its purpose.

Change the name of the method.



Before changing the name, **check if the method overrides (or is overridden) a super class method (by subclasses methods)**. In that case **all the names must change**.

Changing the name implies changing the callers of the method.

What we do if this is not possible ??

We can **keep the previous version** of the method and **make it call the new one to avoid duplication**; mark the old version of the method as **deprecated**

50

Comments

What to Do

When a comment explains **preconditions**, consider using **Introduce Assertion** to replace the comment with code.

51

Introduce Assertion

A section of code assumes something about the state of the program.

Make the assumption explicit with an assertion.

```
public void setup() {
    Connection conn = getConnection();
    // conn should be != null to the
    // rest of the code to work...
    .....
}
```



```
// In Java there is a specific programming construct!!
// Don't confuse it with JUnit
// To enable them java -ea

public void setup() {
    Connection conn = getConnection();
    assert conn != null : "Connection is null";
}
```

52

Introduce Assertion

Often sections of **code** work only if certain **conditions are true**. Such assumptions often are **not stated** but can only be decoded by looking through an algorithm.

Sometimes the assumptions are stated with a **comment**.

A better technique is to make the assumption explicit by **writing an assertion**.

Assertions act as **communication** and **debugging aids**. In **communication** they **help the reader understand the assumptions** the code is making. In **debugging**, assertions can **help catch bugs** closer to their origin.